Veil

Privacy Middleware for Agent Tool Calls

"The agent can work with a tool while Veil controls which sensitive values cross the tool boundary - and the audience can inspect what the tool actually received."

A local-first hackathon prototype. A user sends one synthetic helpdesk message. A local agent (deterministic offline, or optional Gemini) prepares a tool call. Every call passes through the Veil gateway - detect, tokenize, verify - before the offline simulated helpdesk sees it.

Demo rule (read first)

React chat accepts only tickets present in the local helpdesk. The only seeded ticket is TICKET-DEMO-1042. Unknown / missing / malformed IDs are rejected before any agent or tool runs. Use synthetic data only - never real personal data.

Contents: 1 How it works | 2 Where it is used | 3 Components | 4 Privacy scope and limits | 5 Demo screenshots | 6 Results and run instructions

mitk-project | Python 3.12 + FastAPI + React + Streamlit fallback | October 2026

1 How it works

There are three real parts: (A) the agent the user talks to, (B) the Veil middleware in the middle, and (C) the simulated helpdesk tool. The agent can never call the tool directly - the gateway is the only path.

End-to-end flow (one run, one run_id):

```
1. User sends synthetic message with TICKET-DEMO-1042  (React chat /)
2. POST /api/runs -> one run_id; backend picks deterministic or Gemini agent
3. Agent prepares ONE ticket-lookup call (lookup_helpdesk_ticket)
4. Veil scans text + JSON recursively, replaces hits with [[TYPE:token]]
5. Egress gate re-scans the exact outgoing payload; fail-closed on residue
6. Simulated helpdesk captures the EXACT payload it received (ground truth)
7. Helpdesk returns a tokenized reply; Veil restores tokens on trusted path
8. Agent answers; chat shows answer, workflow page shows same-run evidence
```

The key guarantee

The tool only ever sees tokens. The proof panel shows the deep copy captured inside the helpdesk (SimulatedHelpdeskTool.received), never a reconstruction. Blocked runs stop at the egress gate: zero tool calls, zero receipts, no invented answer.

- Tokenize: reversible task-scoped pseudonyms such as [[EMAIL:VA1B2...]]; vault is in-memory and cleared after each run.
- Verify: a second detection pass over the sanitized payload plus tool name; any leftover value blocks dispatch.
- Restore: only the trusted local path back to the agent maps tokens to originals; logs and audit store counts, not values.

2 Where it is used

Built for the Moodlakatte Institute of Technology (CSE) hackathon brief: middleware that intercepts agent-to-tool payloads, detects selected PII in text and structured data, replaces values with reversible tokens, blocks leakage, restores values in trusted responses, and demonstrates detection, leakage prevention, restoration, and processing time.

Brief requirement	How Veil shows it	Where to look
Detect PII (text + JSON)	Recursive scan; detections table per run	Workflow stage 3; benchmark CLI
Prevent leakage	Egress gate; exact tool receipt has 0 originals	'What helpdesk received' panel
Restore answers	Vault restore; tokenized vs restored reply	Workflow stages 5-6
Processing time	Gateway-only p50/p95 latency	Run receipt; benchmark-results.md
Blocked path	Fault-injected miss -> gate blocks, 0 calls	'Show blocked example'

Two-minute judge walkthrough:

- 1. Open the chat, send a made-up request containing TICKET-DEMO-1042 plus synthetic contact values. Stay in deterministic mode for a repeatable run.
- 2. Watch the six stages light up; open 'Show the proof from this workflow' and point at 'What the helpdesk actually received' (tokens, not originals).
- 3. Click 'Show blocked example': the gate stops the run before the helpdesk; downstream stages read 'not reached'.
- 4. In the Streamlit fallback, run the nested-JSON structured payload to show gateway reuse outside the ticket route.
- 5. Quote benchmark only as small synthetic evidence (see page 6), never as universal-PII proof.

Deterministic vs Gemini (optional)

Default = deterministic offline agent (no key, no network, fully repeatable). Gemini mode (VEIL_AGENT_MODE=gemini + server-side GEMINI_API_KEY) sends ONLY a locally tokenized prompt to Google's API, accepts exactly one allowlisted lookup proposal, still routes it through Veil, and restores tokens locally. Tool stays simulated and local. Treat Gemini as 'live only after one fresh run shows every stage through the final answer'.

3 Components used

Python owns privacy and truth; React owns the judge story; Streamlit is the fallback. Both UIs call the same workflow code.

Layer	Files	Job
Detector	veil/detection.py	11 regex/checksum recognizers + overlap priority
Payload	veil/payload.py	Recursive JSON sanitize; egress verify; EgressBlocked
Vault	veil/token_vault.py	Per-task token map; mask/restore/clear
Gateway	veil/gateway.py	Mandatory call_tool boundary + AuditReceipt
Agent (local)	veil/demo_agent.py	Builds 1 ticket lookup; formats answer
Agent (Gemini)	veil/gemini_agent.py	Tokenized prompt; validated proposal; draft
Orchestrator	veil/demo_flow.py	run_ticket_demo + truthful event stages
Tool	veil/helpdesk.py, mock_tool.py	Local helpdesk; deep-copied received receipt
API + runs	veil/api.py, run_registry.py	POST/GET runs; SSE replay; 100-run TTL store
Chat UI	frontend/.../ChatPage.tsx	Single message box; ticket rule; answer card
Workflow UI	frontend/.../WorkflowPage.tsx	6-stage evidence timeline (CSS/SVG + Motion)
Fallback UI	veil/demo.py	Streamlit deterministic + structured-JSON demo
Evaluation	veil/evaluation.py, data/*.json	Dev + held-out scoring; latency; leak scan

Tech stack:

Backend: Python 3.12, FastAPI (>=0.115), Uvicorn, Streamlit (>=1.40), google-genai (>=1.0,<3, optional), python-dotenv. Frontend: React 19 + TypeScript + Vite, React Router 7, Motion 14, Vitest + Testing Library, Playwright. Note: @xyflow/react is installed but intentionally unused - the workflow page uses an accessible vertical rail instead. Tests: pytest (backend), Vitest (frontend). Run store: in-memory, capped at 100 runs, 15-minute expiry.

4 Privacy scope and honest limits

Detector	What it catches (synthetic)	Note
EMAIL / PHONE_NUMBER	RFC-like emails; formatted + labelled phones	Bare 10-digit lookalikes can false-positive
PERSON	Sample lexicon + 'My name is X' + name fields	Least reliable; unknown names missed
AADHAAR / BANK_ACCOUNT	Labelled contexts only (aadhaar, account no.)	Unlabelled numbers ignored by design
UPI_ID / STUDENT_ID	VPA handles; STU-DEMO-5digits	Narrow demo shapes
PAYMENT_CARD / IBAN	Luhn-valid cards; mod-97 IBAN (shortest match)	Checksum gates cut false alarms
PAN / IFSC	5L+4D+1L; 4L+0+6AN	Strict demo patterns
CUSTOM	Exact user-listed literals	Covers anything the regexes miss

- Tokens are reversible pseudonyms, NOT encryption. Anyone holding the vault map can reverse them - that is why the vault lives only for the active run and is cleared in a finally block.
- Fail-closed: residual detection, detector exception, bad tool name, or tool error all block before dispatch. A blocked trace never shows a helpdesk receipt.
- Never claim universal detection, production security, or encryption. Image/audio, other languages, and obfuscation are out of scope.
- Synthetic data only. Server logs record run IDs, counts, and timings - never raw values. The UI may show the judge's own synthetic input for the local story.

5 Demo screenshots (paste your captures here)

Run the app with the commands on the last page, capture each view, and paste over the gray boxes. Keep the captions - they tell the judge what to notice.

Shot 1 - Chat: valid ticket accepted

[Paste screenshot here]

Chat (/) after sending a synthetic TICKET-DEMO-1042 message: agent answer card + 'View animated workflow' link. Badge reads Offline local agent (or Gemini agent in Gemini mode).

- Message contains TICKET-DEMO-1042 and synthetic values
- Answer is the real agent answer for that run_id

Shot 2 - Workflow: six connected stages

[Paste screenshot here]

Workflow (/runs/{run_id}): User -> Agent -> Veil -> Egress gate -> Helpdesk -> Restore -> Answer. Same run_id as the chat; egress-verified indicator visible.

- All six nodes complete on success path
- Token strings [[TYPE:V...]] highlighted

Demo screenshots (continued)

Shot 3 - Proof: what the helpdesk actually received

[Paste screenshot here]

Expanded proof panel: the EXACT captured payload (tokens, ticket ID intact), the tokenized tool reply, and Veil's restored reply side by side.

- No original contact values in the captured payload
- Restored reply matches the synthetic originals

Shot 4 - Blocked path + structured example

[Paste screenshot here]

Blocked example: gate marked blocked, helpdesk and later stages 'not reached', 0 tool calls. Optionally add the Streamlit nested-JSON receipt beside it.

- Blocked run shows a reason, not a fabricated answer
- Structured demo shows nested receipt with tokens

6 Results and how to run it

Latest recorded synthetic benchmark (2026-10-10, Python 3.12.14, Windows). Timings vary by machine - re-run on the judging laptop. Small fixed synthetic corpus: strong signal, not a general PII claim.

Split	Cases / spans	Precision / Recall / F2
Development	33 / 49 planted	96.08% / 100% / 99.19%
Held-out	26 / 26 planted	92.86% / 100% / 98.48%

Known trade-off: 2 phone false positives per split from benign numeric lookalikes (documented in benchmark-results.md). Latency method: 20 warm-ups + 100 measured gateway-only calls per case (tool, restore, network excluded).

Run it on Windows (two terminals, project root):

```
# Terminal 1 - API:
.\.venv\Scripts\python.exe -m uvicorn veil.api:app --host 127.0.0.1 --port 8000
# Terminal 2 - React UI:
cd frontend
npm ci
npm run dev
# Open http://127.0.0.1:5173/  ->  Send ticket message  ->  View animated workflow
# Optional Streamlit:  .\.venv\Scripts\python.exe -m streamlit run veil/demo.py
# Benchmark:  .\.venv\Scripts\python.exe -m veil.evaluation
```

What to say in closing

Veil proves one thing well: the agent did its job while the tool never saw the originals - and you can check the exact receipt yourself. Detection is selected-pattern, tokens are pseudonyms, and every number here is synthetic evidence. Thank you.